



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**
título del TFG



Presentado por Rebeca Cuesta Estépar
en Universidad de Burgos — 31 de enero
de 2026

Tutor: nombre tutor



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. nombre tutor, profesor del departamento de nombre departamento, área de nombre área.

Expone:

Que el alumno D. Rebeca Cuesta Estépar, con DNI dni, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado título de TFG.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 31 de enero de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. nombre tutor

D. nombre co-tutor

Resumen

En este primer apartado se hace una **breve** presentación del tema que se aborda en el proyecto.

Descriptores

Palabras separadas por comas que identifiquen el contenido del proyecto Ej: servidor web, buscador de vuelos, android ...

Abstract

A **brief** presentation of the topic addressed in the project.

Keywords

keywords separated by commas.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	3
3. Conceptos teóricos	5
3.1. Secciones	5
3.2. Referencias	5
3.3. Imágenes	6
3.4. Listas de items	6
3.5. Tablas	7
4. Técnicas y herramientas	9
4.1. Lenguajes y entornos de programación	9
4.2. Base de datos y despliegue	9
4.3. Frameworks	10
4.4. Bibliotecas	13
4.5. Otras herramientas y técnicas	13
5. Aspectos relevantes del desarrollo del proyecto	15
6. Trabajos relacionados	17

7. Conclusiones y Líneas de trabajo futuras	19
Bibliografía	21

Índice de figuras

3.1. Autómata para una expresión vacía	6
--	---

Índice de tablas

3.1. Herramientas y tecnologías utilizadas en cada parte del proyecto	7
---	---

1. Introducción

Descripción del contenido del trabajo y del estructura de la memoria y del resto de materiales entregados.

2. Objetivos del proyecto

Este apartado explica de forma precisa y concisa cuales son los objetivos que se persiguen con la realización del proyecto. Se puede distinguir entre los objetivos marcados por los requisitos del software a construir y los objetivos de carácter técnico que plantea a la hora de llevar a la práctica el proyecto.

3. Conceptos teóricos

En aquellos proyectos que necesiten para su comprensión y desarrollo de unos conceptos teóricos de una determinada materia o de un determinado dominio de conocimiento, debe existir un apartado que sintetice dichos conceptos.

Algunos conceptos teóricos de L^AT_EX¹.

3.1. Secciones

Las secciones se incluyen con el comando `section`.

Subsecciones

Además de secciones tenemos subsecciones.

Subsubsecciones

Y subsecciones.

3.2. Referencias

Las referencias se incluyen en el texto usando `cite [?]`. Para citar webs, artículos o libros `[?]`, si se desean citar más de uno en el mismo lugar `[?, ?]`.

¹Créditos a los proyectos de Álvaro López Cantero: Configurador de Presupuestos y Roberto Izquierdo Amo: PLQuiz

3.3. Imágenes

Se pueden incluir imágenes con los comandos standard de $\text{\LaTeX}$, pero esta plantilla dispone de comandos propios como por ejemplo el siguiente:



Figura 3.1: Autómata para una expresión vacía

3.4. Listas de items

Existen tres posibilidades:

- primer item.
- segundo item.

1. primer item.
2. segundo item.

Primer item más información sobre el primer item.

Segundo item más información sobre el segundo item.

■

Herramientas	App	AngularJS	API REST	BD	Memoria
HTML5		X			
CSS3		X			
BOOTSTRAP		X			
JavaScript		X			
AngularJS		X			
Bower		X			
PHP			X		
Karma + Jasmine		X			
Slim framework			X		
Idiorm			X		
Composer			X		
JSON		X	X		
PhpStorm		X	X		
MySQL				X	
PhpMyAdmin				X	
Git + BitBucket		X	X	X	X
MikTeX					X
TeXMaker					X
Astah					X
Balsamiq Mockups		X			
VersionOne		X	X	X	X

Tabla 3.1: Herramientas y tecnologías utilizadas en cada parte del proyecto

3.5. Tablas

Igualmente se pueden usar los comandos específicos de $\text{\LaTeX}$ o bien usar alguno de los comandos de la plantilla.

4. Técnicas y herramientas

Esta parte de la memoria tiene como objetivo presentar las técnicas metodológicas y las herramientas de desarrollo que se han utilizado para llevar a cabo el proyecto. Si se han estudiado diferentes alternativas de metodologías, herramientas, bibliotecas se puede hacer un resumen de los aspectos más destacados de cada alternativa, incluyendo comparativas entre las distintas opciones y una justificación de las elecciones realizadas. No se pretende que este apartado se convierta en un capítulo de un libro dedicado a cada una de las alternativas, sino comentar los aspectos más destacados de cada opción, con un repaso somero a los fundamentos esenciales y referencias bibliográficas para que el lector pueda ampliar su conocimiento sobre el tema.

4.1. Lenguajes y entornos de programación

Python

Entorno virtual

VisualStudio

4.2. Base de datos y despliegue

Sistema gestor de base de datos: MySQL

La aplicación requiere de una base de datos para almacenar y consultar toda la información relacionada horarios, reservas, y aulas. Para llevarlo a cabo he elegido MySQL como Sistema Gestor de Base de Datos (SGBD),

por su amplio uso y por tener compatibilidad directa con Django mediante su backend. Y Django documenta soporte oficial para MySQL a partir de versiones modernas.

Con lo que he elegido MySQL ya que a nivel de mantenimiento permite un control independiente de la versión del sistema gestor de bases de datos, evitando que me limite la compatibilidad con versiones del framework del backend elegido.

Servidor y entorno de ejecución: Apache

Alternativa evaluada: XAMPP

Como alternativa se valoró XAMPP, que es un paquete que tiene todo en uno, es decir, te proporciona el mismo programa el sistema gestor de base de datos y el servidor web, incluyendo Apache y MariaDB. Sin embargo, XAMPP distribuye versiones concretas de sus componentes, lo que limitaba el desfase del proyecto ya que las nuevas versiones de Django requieren versiones del sistema gestor de base de datos más actualizadas para poder tener compatibilidad. Por lo que, haber escogido XAMPP como opción, hubiese tenido que instalar versiones de Django anteriores, lo que aceleraría el proceso de desfase de la aplicación.

4.3. Frameworks

Backend: Django ((Django REST Framework))

Una vez seleccionado Python como lenguaje principal del proyecto, decidí utilizar Django como framework de desarrollo para el backend. La elección fue tomada principalmente por la necesidad de una arquitectura modular, mantenible y escalable, ya que el sistema debía organizarse en componentes independientes (por ejemplo, gestión de horarios, gestión de aulas, reservas, calendario, etc.) con responsabilidades bien delimitadas, y por ser un proyecto basado en el desarrollo de una aplicación web.

Django favorece este enfoque mediante su organización basada en aplicaciones, permitiendo dividir el proyecto en módulos reutilizables e independientes, con el fin de dividir la lógica del proyecto, facilitando el crecimiento del mismo y reduciendo el acoplamiento entre distintas funcionalidades.

Además, Django se apoya en el patrón arquitectónico MVT (Modelo-Vista-Plantilla), que guía la separación de responsabilidades:

- **Modelo (Model):** define la estructura de datos y la interacción con la base de datos mediante el ORM.
- **Vista (View):** contiene la lógica de negocio y gestiona las peticiones/respuestas, es decir, procesa las solicitudes que entran por parte de los usuarios y
- **Plantilla (Template):** se encarga de la presentación de los datos al usuario; en este proyecto, esta separación sigue siendo útil aunque el frontend se implemente con React, ya que el backend se centra en exponer servicios y lógica.

Backend: Django REST Framework

Dado que vamos a necesitar que el frontend consuma datos del backend, concluimos que iba a ser necesaria la utilización de una API, mediante endpoints. Por ello decidió instalarse Django REST Framework. Esta elección se debe a que DRF proporciona componentes

Esta elección se debe a que DRF proporciona componentes esenciales para este tipo de arquitectura. Entre los más relevantes destacan:

- **Serializadores:** permiten transformar instancias de modelos y consultas en formatos intercambiables (por ejemplo JSON) y, además, validar los datos de entrada antes de persistirlos.
- **Vistas y ViewSets:** facilitan la implementación de endpoints reutilizables y coherentes, reduciendo código repetido y estandarizando operaciones habituales (listar, crear, modificar y eliminar recursos).
- **Autenticación y permisos:** permiten controlar el acceso a la API, aplicando políticas de seguridad según el tipo de usuario o rol.

Frontend: React + Vite

Para llevar a cabo el desarrollo del frontend, tuve en cuenta dos opciones: React y Angular.

Aunque verdaderamente React se define como una librería para construir interfaces de usuarios, que se basa en componentes, permitiendo reutilizar piezas y separar de manera clara las funcionalidades de la interfaz. Además, nos permite una mayor flexibilidad de cara a la estructura del proyecto, permitiendo seleccionar únicamente las dependencias necesarias. Por otro

lado, tenemos Angular un framework que es más cerrado, con decisiones y herramientas integradas, teniendo una mayor carga inicial y la adaptación a la forma de trabajo es menos flexible.

Con lo que finalmente elegí utilizar React, porque me pareció importante la característica de la flexibilidad. Una vez seleccionado React, investigué un poco más a fondo y encontré dos formas de iniciarlo en el proyecto: Create React App y Vite + React. Tras buscar información relativa a ambas opciones, finalmente elegía la opción que contiene Vite.

Y es que Vite es una alternativa más moderna, y es que de hecho el propio equipo de React no recomienda actualmente su utilización debido a limitaciones, y recomienda utilizar herramientas integradas, como es el caso de la escogida.

Esta herramienta destaca por el hecho de que proporciona un entorno de desarrollo ligero y rápido al contar con un servidor de desarrollo optimizado y capacidades como la recarga en caliente (HMR - Hot Module Replacement) que es instantáneo sin importar lo grande que sea la app, lo que mejora el flujo de trabajo durante la implementación. Además, Vite utiliza Rollup que es muy rápido en eliminar el código muerto que no usas, resultando en una aplicación final más ligera mejorando las métricas de rendimiento web.

CSS: Tailwind

Para la estética del frontend valoré las dos siguientes alternativas: Tailwind CSS y Bootstrap.

Busqué documentación sobre ambos frameworks. Bootstrap permite crear interfaces de forma más rápida debido a que cuenta con un catálogo de componentes predefinidos (botones, formularios,...), pero debido a esta característica en muchas ocasiones las aplicaciones que hacen uso de este framework utilizando su configuración por defecto tienen una estética muy parecida. Se puede personalizar pero requiere estar sobrescribiendo. Por otro lado, tenemos Tailwind CSS es *utility-first*, que se basa en clases atómicas que se combinan para construir diseños a medida. Este enfoque permite una mayor flexibilidad y no depende de estilos preestablecidos, además encaja con el desarrollo por componentes del frontend. Por todos estos motivos, fue por los que elegí este framework para los estilos del proyecto.

4.4. Bibliotecas

Biblioteca de Calendarios: React Big Calendar

4.5. Otras herramientas y técnicas

Scrum

Github

Zenhub

TeXstudio

5. Aspectos relevantes del desarrollo del proyecto

Este apartado pretende recoger los aspectos más interesantes del desarrollo del proyecto, comentados por los autores del mismo. Debe incluir desde la exposición del ciclo de vida utilizado, hasta los detalles de mayor relevancia de las fases de análisis, diseño e implementación. Se busca que no sea una mera operación de copiar y pegar diagramas y extractos del código fuente, sino que realmente se justifiquen los caminos de solución que se han tomado, especialmente aquellos que no sean triviales. Puede ser el lugar más adecuado para documentar los aspectos más interesantes del diseño y de la implementación, con un mayor hincapié en aspectos tales como el tipo de arquitectura elegido, los índices de las tablas de la base de datos, normalización y desnormalización, distribución en ficheros³, reglas de negocio dentro de las bases de datos (EDVHV GH GDWRV DFWLYDV), aspectos de desarrollo relacionados con el WWW... Este apartado, debe convertirse en el resumen de la experiencia práctica del proyecto, y por sí mismo justifica que la memoria se convierta en un documento útil, fuente de referencia para los autores, los tutores y futuros alumnos.

6. Trabajos relacionados

Este apartado sería parecido a un estado del arte de una tesis o tesina. En un trabajo final grado no parece obligada su presencia, aunque se puede dejar a juicio del tutor el incluir un pequeño resumen comentado de los trabajos y proyectos ya realizados en el campo del proyecto en curso.

7. Conclusiones y Líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo. Éstas pueden ser de diferente índole, dependiendo de la tipología del proyecto, pero normalmente van a estar presentes un conjunto de conclusiones relacionadas con los resultados del proyecto y un conjunto de conclusiones técnicas. Además, resulta muy útil realizar un informe crítico indicando cómo se puede mejorar el proyecto, o cómo se puede continuar trabajando en la línea del proyecto realizado.

Bibliografía
